# Python Logging

(*) | VVV imp

(*) >>> import logging

```
>>> logging.basicConfig(level=logging.DEBUG)
logging.debug('This is a debug mess.')
logging.info('.....')
logging.warning('.....')
logging.error('.....')
logging.critical('.....')
```

log messages

default levels (*)

## configure logging

```
>>> logging.basicConfig(
        filename='app.log',
        filemode='w',
        level=logging.DEBUG,
        format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
        datefmt='%Y-%m-%d %H:%M:%S'
    )
```

('a', 'w')

basicConfig setting for logging..

run first.

(*) Create logger.py.
→ write all log code

# Logging

The level decides which logs to keep, which to ignore

**Step 1** → Create a logger.py file, with the basic Config settings

**Step 2** → in logger.py, add →

```
>>> import logging
logging.basicConfig(filename='app.log',
                    filemode='a',
                    level=logging.DEBUG,
                    format=format_1,
                    datefmt=format_2)

format_1 = '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
format_2 = '%Y-%m-%d %H:%M:%S'
```

filemodes 'a' or 'w'
↓ ↓
append overwrite

**Step 3** → in code file,

→ from logger import logging

**Step 4** → Whenever log needed,

logging.debug('.....')

# Multiple Loggers

# logger for ~~A~~ a single module 1

names of loggers modules

```
>>> logger1 = logging.getLogger('m1')
    logger1.setLevel(logging.DEBUG)
```

These go into the modules

```
>>> logger2 = logging.getLogger('m2')
    logger2.setLevel(logging.ERROR)
```

# Basic config settings.

```
>>> logging.basicConfig(level = logging.DEBUG,
                        format = form.
                        datefmt = for..
                        )
```

Now,

```
logger1.debug('msg1')
logger2.debug('msg2')
```

Logging levels (threat / priority)

DEBUG < INFO < WARNING

< ERROR < CRITICAL

# # More logging settings

```
>>> logging.basicConfig(
        level = logging.DEBUG,
        format = f...
        datefmt = f...
        ~~handlers~~
        handlers = [logging.FileHandler('appl.log'),
                    logging.StreamHandler()
                   ]
    )
```

can also put mode

This displays on console as well

replaces the explicit filename = ...
& filemode = ''

---

# # Removing existing handlers

```
>>> for handler in logging.root.handlers[:]:
        logging.root.removeHandler(handler)
```

---

(*) → Go through logging assignment for more details.